

CI/CD Threat Model

Enterprise DevSecOps Security Lab

A professional threat modeling document for the GitHub Actions pipeline, container build process, artifact registry, GitOps deployment path, Kubernetes delivery workflow and associated security controls.

Author Jagriti Banerjee	Document Type CI/CD Threat Model
Project Enterprise DevSecOps Security Lab	Environment Private VM lab using Docker, Kind, GitHub Actions, GHCR, ArgoCD and Kubernetes
Primary Focus Pipeline abuse, runner compromise, dependency risk, artifact tampering, credential misuse and GitOps drift	Assessment Style Defensive security engineering and portfolio-grade technical documentation

CI/CD Threat Model - Pipeline Trust Boundaries

Source control, workflow execution, artifact registry, GitOps deployment and Kubernetes runtime boundaries.

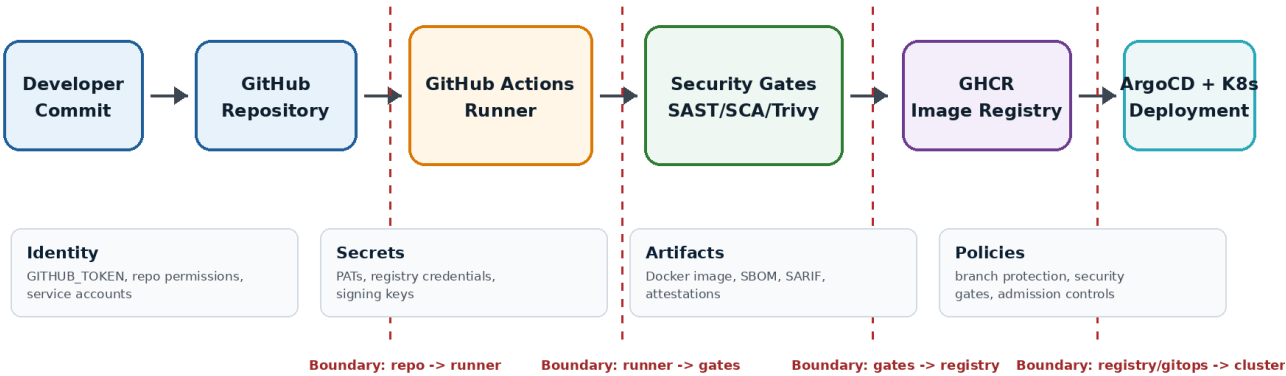


Figure 1 - CI/CD pipeline trust boundaries and high-level threat model context.

Executive positioning
This document explains what can go wrong in the software delivery pipeline, which assets are at risk, how those risks were modeled, and which preventive, detective and corrective controls were implemented in the Enterprise DevSecOps lab.

1. Executive Summary

This CI/CD threat model documents the security risks and controls associated with the Enterprise DevSecOps project delivery pipeline. The modeled pipeline starts at source code commit and continues through GitHub Actions workflow execution, static application security testing, dependency scanning, container image build, container scanning, registry publishing, image signing, SBOM generation, ArgoCD GitOps deployment, Kubernetes admission control and runtime monitoring.

The threat model is written from a defender perspective. It identifies likely attack paths, trust boundaries, misuse cases, security controls, residual risk and evidence captured during the lab. The goal is to demonstrate practical security engineering capability rather than only describing tools.

Area	Summary
Threat model scope	Source repository, CI/CD workflow, GitHub-hosted runner, dependency resolution, Docker build, GHCR registry, ArgoCD deployment, Kubernetes admission and runtime telemetry.
Critical assets	Source code, pipeline workflow file, secrets, GITHUB_TOKEN permissions, container image, SBOM, signing key, Kubernetes manifests, ArgoCD application state and runtime evidence.
Primary risk themes	Credential abuse, malicious workflow modification, vulnerable dependencies, build tampering, artifact substitution, registry misuse, GitOps drift and weak admission enforcement.
Implemented controls	SAST, SCA, Trivy scanning, SARIF reporting, limited workflow permissions, Docker hardening, GHCR push, Cosign signing, Syft SBOM, ArgoCD self-healing, Gatekeeper policy and Falco observability.
Retest outcome expected	Security gates identify high-risk issues, deployment drift is corrected, signed artifacts are verifiable, and runtime anomalies produce evidence for investigation.

Conclusion: The current lab pipeline has strong defensive coverage for application security, dependency risk, container scanning, supply-chain evidence and GitOps control. The highest residual risks are around workflow permission minimization, secret governance, branch protection, third-party action pinning and policy enforcement maturity.

2. Threat Modeling Methodology

The threat model follows a practical engineering approach based on asset identification, trust boundary analysis, STRIDE-style threat enumeration, attack path mapping, risk scoring and control validation. Each threat is assessed in the context of the lab pipeline and mapped to recommended preventive, detective and corrective controls.

Step	Activity	Output
1	Define the CI/CD system and identify pipeline stages.	Pipeline context diagram and scope statement.
2	Identify assets, identities, secrets, artifacts and trust boundaries.	Asset inventory and boundary map.
3	Enumerate threats using STRIDE categories and CI/CD-specific abuse cases.	Threat register with scenario descriptions.
4	Score each threat using likelihood, impact and exploitability.	Prioritized risk ranking.
5	Map controls to threats across prevention, detection and response.	Control matrix and remediation plan.
6	Validate controls using lab evidence and retest commands.	Evidence-backed threat model closure.

2.1 Risk Scoring Method

Risk is scored qualitatively using likelihood and impact. Likelihood considers attacker access, required privileges, complexity and exposure. Impact considers confidentiality, integrity, availability, build integrity, deployment integrity and auditability.

Rating	Likelihood Meaning	Impact Meaning
Critical	Highly plausible with limited friction or common misconfiguration.	Could enable artifact compromise, unauthorized deployment, secret theft or broad cluster impact.
High	Plausible when an attacker gains contributor, workflow or token access.	Could compromise one or more major pipeline stages or sensitive assets.
Medium	Requires specific access, timing or chaining.	Could weaken assurance, delay detection or enable limited tampering.
Low	Requires unusual conditions or substantial access.	Limited direct impact but useful for hardening.

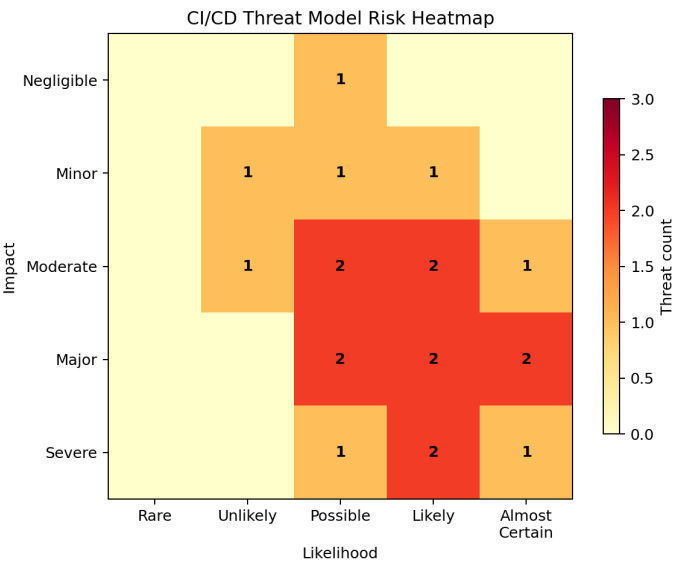


Figure 2 - Risk heatmap based on modeled CI/CD threat scenarios.

3. System Context and Pipeline Architecture

The pipeline is modeled as a sequence of technical trust zones. The most important concept is that every transition from one zone to another creates an opportunity for abuse if identity, integrity and authorization are not controlled.

CI/CD Threat Model - Pipeline Trust Boundaries

Source control, workflow execution, artifact registry, GitOps deployment and Kubernetes runtime boundaries.

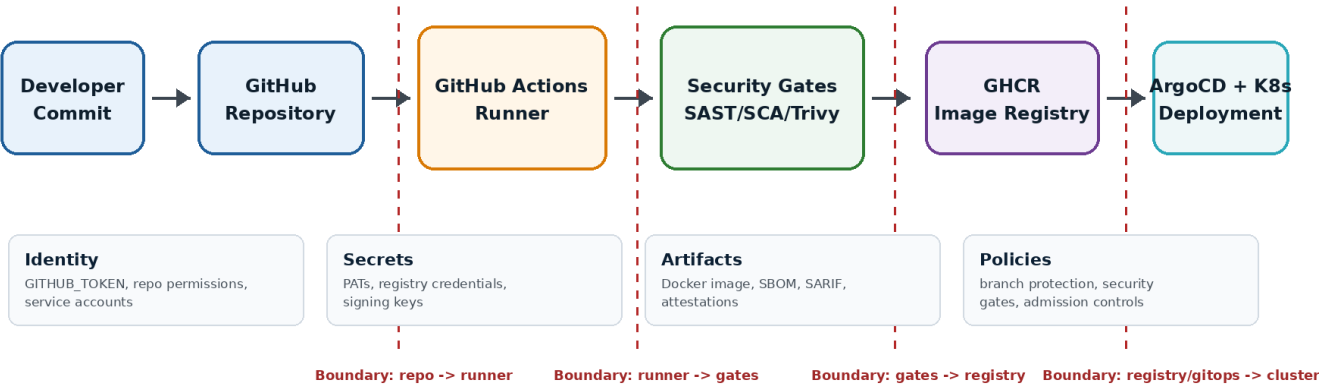


Figure 3 - CI/CD architecture and trust boundary model.

Pipeline Stage	Purpose	Trust Boundary / Security Concern
Source repository	Stores application code, Dockerfile, Kubernetes manifests and workflow YAML.	Malicious commits, branch bypass, workflow tampering and secret leakage.
GitHub Actions runner	Executes SAST, SCA, image build and scanning jobs.	Untrusted code execution, runner escape, token misuse and dependency poisoning.
Security gates	Run Bandit, pip-audit and Trivy checks.	Gate bypass, non-blocking scans, weak thresholds and artifact manipulation.
GHCR registry	Stores Docker image artifacts.	Unauthorized image push, tag overwrite and image substitution.
Supply chain layer	Cosign signatures, SBOMs and provenance evidence.	Unsigned images, unverifiable artifacts and missing SBOM lineage.
ArgoCD / Kubernetes	Deploys hardened manifests from Git into Kind cluster.	GitOps drift, excessive permissions, weak admission policy and runtime compromise.

4. Critical Asset Inventory

The following assets were identified as high-value pipeline security targets. Protecting these assets is essential because CI/CD compromise can lead to downstream application, container and cluster compromise.

Asset	Why It Matters	Threat Examples	Control Expectations
Workflow YAML	Controls what executes during CI/CD.	Malicious step injection, disabled scans, altered image tags.	Code review, branch protection, least privilege, signed commits where possible.
GITHUB_TOKEN	Authenticates workflow jobs to GitHub APIs and registry operations.	Package overwrite, SARIF misuse, repo content access.	Minimal permissions, job-level permission scoping, no broad write defaults.
Repository secrets	May contain PATs, tokens or signing material.	Secret exfiltration via PR, logs or malicious workflow.	Secret scanning, environment protection, masked output, restricted access.
Dependency lock/requirements files	Define third-party software entering the build.	Dependency confusion, vulnerable version, typosquatting.	pip-audit, pinning, review of version changes, private index controls.
Docker image	Deployable runtime artifact.	Tampering, vulnerable base image, injected malicious binary.	Trivy, non-root image, Cosign signature, SBOM.
GHCR image tags	Used by deployment systems to pull images.	Tag overwrite, downgrade or substitution.	Immutable digest references, signing verification, controlled push.
Kubernetes manifests	Define deployed permissions and runtime posture.	Privileged pods, hostPath, weak RBAC, open networking.	Trivy config, Gatekeeper, Pod Security, review and GitOps.
ArgoCD Application	Controls sync from Git to cluster.	Drift, unauthorized repo path, broad project permissions.	Self-healing, restricted repos, ArgoCD RBAC, audit review.

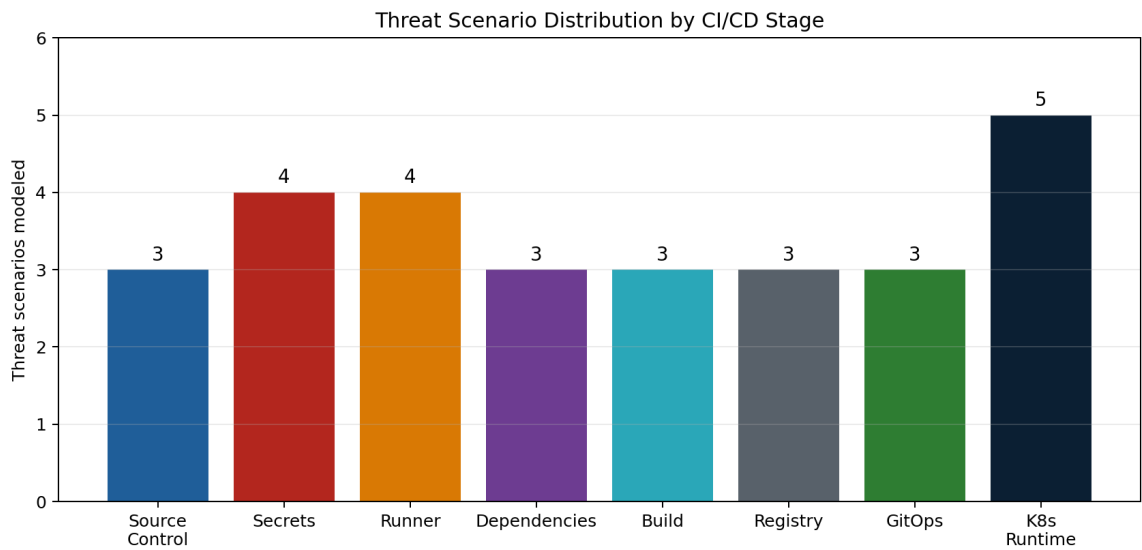


Figure 4 - Threat distribution across modeled CI/CD stages.

5. Trust Boundaries and Assumptions

The pipeline contains several trust boundaries. A threat model is most useful when these boundaries are explicit, because security controls must be placed before and after boundary crossings.

Boundary	Transition	Threat Modeling Concern	Control Placement
B1	Developer workstation to GitHub repository	A malicious or compromised commit can modify application code, Dockerfile or workflow logic.	Branch protections, review, SAST, secret scanning.
B2	GitHub repository to Actions runner	Runner executes code from the repository and may receive tokens/secrets.	Least privilege permissions, safe PR workflow design, controlled secrets.
B3	Actions runner to dependency ecosystem	Build pulls packages and actions from external sources.	Version pinning, SCA, action pinning, allowlists.
B4	Runner to GHCR registry	Build output becomes deployable artifact.	Registry auth, controlled tags, image scanning, signing.
B5	GHCR/Git repo to ArgoCD	Deployment controller pulls desired state and artifacts.	Repo allowlist, GitOps policy, ArgoCD RBAC, digest references.
B6	Kubernetes admission to runtime	Workload becomes active in cluster.	Gatekeeper, Pod Security, RBAC, NetworkPolicy, Falco.

Key assumption

The lab is intentionally private and controlled. The threat scenarios are modeled as defensive exercises to demonstrate how CI/CD weaknesses could be abused and how the pipeline should be hardened before production use.

6. STRIDE Mapping for CI/CD

The STRIDE model is used to ensure the pipeline is reviewed across identity, integrity, auditability, confidentiality, availability and authorization concerns.

STRIDE Category	Pipeline Threat Example	Security Control
Spoofing	Attacker uses stolen token or compromised service account to push image or modify deployment.	Least privilege token permissions, protected environments, registry authentication review.
Tampering	Workflow YAML altered to skip scanning or push a malicious image.	Code review, branch protection, security-gate enforcement, GitOps drift correction.
Repudiation	Pipeline changes cannot be traced to a user or commit.	Git history, workflow logs, artifact metadata, signed attestations.
Information Disclosure	Secrets exposed in logs, artifacts or malicious pull request workflow.	Secret masking, restricted secrets, log review, secret scanning.
Denial of Service	Runner, registry or cluster resource exhaustion blocks deployment.	Resource limits, job timeouts, monitoring, retry policy, alerting.
Elevation of Privilege	Workflow token or Kubernetes service account receives broad write or cluster-admin access.	Minimum permissions, scoped service accounts, RBAC audit, admission controls.

Interpretation: The most significant CI/CD threats in this lab are tampering and elevation of privilege because a compromised workflow or deployment identity can alter build outputs and influence runtime state.

7. Threat Register

This register summarizes the main modeled threats. The risk values are designed for lab reporting and prioritization. They can be adjusted in a production environment based on exposure, branch protection maturity, organization access model and asset criticality.

ID	Threat Scenario	Stage	Risk	Primary Mitigation
TM-01	Malicious workflow change disables Bandit, pip-audit or Trivy scans.	Source / CI	High	Branch protection, review workflow files, required checks, CODEOWNERS.
TM-02	GITHUB_TOKEN has more permissions than required for the job.	CI identity	High	Set explicit permissions and use job-level minimum scopes.
TM-03	Secret or PAT is exposed through logs, artifacts or unsafe script output.	Secrets	Critical	Secret scanning, masking, environment protection, no echoing secrets.
TM-04	Unpinned third-party GitHub Action changes behavior unexpectedly.	Runner	Medium	Pin actions to full commit SHA or trusted versions, monitor updates.
TM-05	Vulnerable Python dependency enters build through requirements.txt.	SCA	High	pip-audit, version pinning, dependency review and upgrade workflow.
TM-06	Docker image built with vulnerable base image or insecure runtime user.	Build	High	Trivy image scan, non-root Dockerfile, least privilege runtime.
TM-07	Container image tag is overwritten or substituted in registry.	GHCR	High	Use immutable digests, signature verification and controlled push permissions.
TM-08	Unsigned or unverified artifact is deployed.	Supply chain	High	Cosign signing, SBOM and provenance attestations.
TM-09	ArgoCD sync path points to unauthorized manifests or wrong branch.	GitOps	Medium	Restrict repo/path/project permissions and review Application definitions.
TM-10	Manual cluster change creates drift from Git-defined state.	GitOps / K8s	Medium	ArgoCD self-heal, prune, audit alerts.
TM-11	Kubernetes manifest deploys privileged pod or broad RBAC.	K8s admission	Critical	Gatekeeper, Pod Security, RBAC checks, Trivy config scan.
TM-12	Runtime attack occurs after deployment and is not detected.	Runtime	High	Falco detection, Prometheus metrics, Grafana dashboard and incident workflow.

8. Stage-by-Stage Threat Analysis

8.1 Source Repository and Workflow Definition

The repository stores application code, security reports, Kubernetes manifests, Dockerfile and the GitHub Actions workflow. This makes source control one of the most important control points in the pipeline.

Threat	Attack Path	Impact	Control
Workflow tampering	Attacker modifies .github/workflows/ci-pipeline.yml to bypass scans or add malicious commands.	Malicious code can be built and published without detection.	Protect main branch, require review for workflow changes, configure required checks.
Dockerfile tampering	Attacker modifies Dockerfile to add backdoor, run as root or pull malicious base.	Compromised container image and runtime execution.	Dockerfile review, Trivy config, non-root policy and build review.
Manifest tampering	Attacker changes Kubernetes YAML to add privileged securityContext or broad RBAC.	Cluster compromise or lateral movement.	Gatekeeper policies, ArgoCD review, Kubernetes manifest scanning.

8.2 Pipeline Execution and Runner Abuse

GitHub Actions jobs execute repository-defined commands. A compromised workflow step can access job context, workspace files, artifacts and any tokens made available to the job. The threat model therefore treats the runner as a sensitive execution boundary.

Control Area	Current Lab Evidence	Recommended Hardening
SAST job	Bandit scan job exists and uploads report artifacts.	Fail on high-confidence critical findings after training mode is complete.
SCA job	pip-audit scans requirements.txt and uploads JSON report.	Set production threshold to fail on high/critical fixable advisories.
Image scan	Trivy scans Docker image and emits table/SARIF output.	Block critical vulnerabilities unless documented exception exists.
Permissions	Workflow uses explicit contents, security-events, packages and actions permissions.	Reduce permissions per job and avoid broad workflow-level write if not needed.

Relevant workflow security configuration excerpt

```
permissions:
  contents: read
  security-events: write
  packages: write
  actions: read

jobs:
  sast:
    name: SAST - Bandit Python Security Scan
    dependency-scan:
      name: SCA - Python Dependency Scan
    docker-build-and-scan:
      needs: [sast, dependency-scan, filesystem-scan]
```

9. Dependency, Build and Artifact Threats

Dependency and build threats affect the software supply chain before deployment. A pipeline can pass source-level tests but still produce an unsafe artifact if dependencies, base images, build scripts or registry tags are compromised.

Threat Area	Risk Description	Control Implemented	Improvement Opportunity
Dependency vulnerability	Old packages can introduce known security weaknesses.	pip-audit scans Python dependencies.	Introduce dependency review and upgrade SLAs.
Dependency confusion	Build may pull similarly named malicious package.	requirements.txt scanning and controlled lab dependencies.	Use private package index controls and trusted sources.
Build script tampering	Malicious build step can alter image contents.	Docker build is visible in workflow and scan follows build.	Require review for Dockerfile and workflow changes.
Image vulnerability	Base image or packages contain CVEs.	Trivy image scan and SARIF generation.	Enforce fail thresholds for critical fixable CVEs.
Image substitution	Attacker pushes different image to same tag.	GHCR publishing and Cosign signing in supply-chain phase.	Deploy by digest and enforce signature verification at admission.

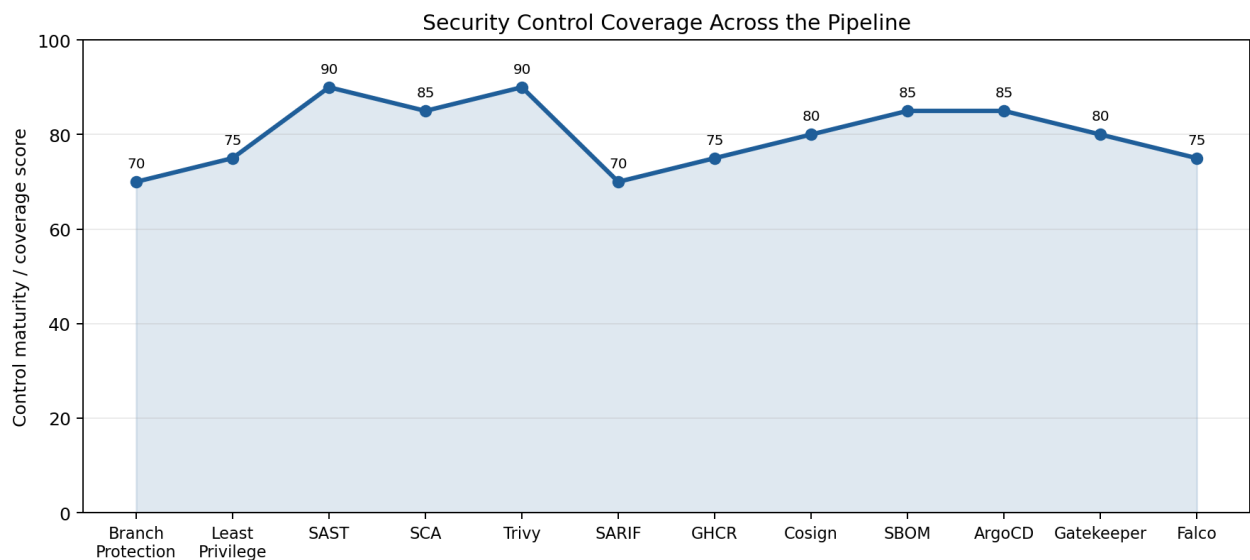
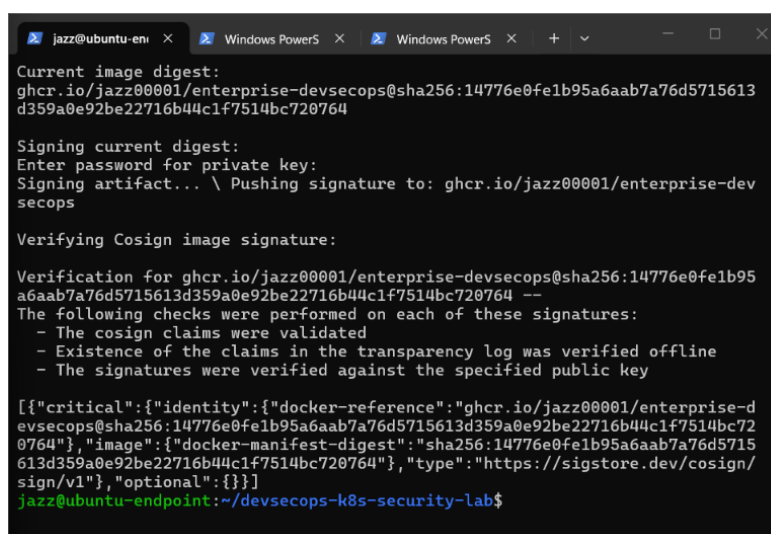


Figure 5 - Security controls mapped across the pipeline.

10. Registry and Supply Chain Threats

Once an image is published to GHCR, it becomes a deployable artifact. Registry compromise, tag overwrite, missing signature verification or weak artifact traceability can cause a secure source repository to produce an unsafe runtime environment.

Control	Purpose	Threat Reduced	Evidence Source
GHCR image push	Publishes the built image to a controlled registry.	Uncontrolled artifact distribution and manual image sharing.	Screenshot 066 - ghcr image pushed.
Cosign signature	Provides cryptographic proof that the image was signed by the lab key.	Image tampering and unsigned artifact deployment.	Screenshot 115 - cosign signature verified.
Syft SBOM	Documents packages and components inside the image.	Poor dependency visibility and weak auditability.	SBOM generation evidence in supply-chain phase.
Provenance attestation	Links image to repository, commit and build context.	Loss of build lineage and weak artifact accountability.	SLSA-style provenance evidence.



```

jazz@ubuntu-enu x Windows PowerShell x Windows PowerShell x + v - □ ×
Current image digest:
ghcr.io/jazz00001/enterprise-devsecops@sha256:14776e0fe1b95a6aab7a76d5715613
d359a0e92be22716b44c1f7514bc720764

Signing current digest:
Enter password for private key:
Signing artifact... \ Pushing signature to: ghcr.io/jazz00001/enterprise-dev
secops

Verifying Cosign image signature:

Verification for ghcr.io/jazz00001/enterprise-devsecops@sha256:14776e0fe1b95
a6aab7a76d5715613d359a0e92be22716b44c1f7514bc720764 --
The following checks were performed on each of these signatures:
- The cosign claims were validated
- Existence of the claims in the transparency log was verified offline
- The signatures were verified against the specified public key

[{"critical":{"identity":{"docker-reference":"ghcr.io/jazz00001/enterprise-d
evsecops@sha256:14776e0fe1b95a6aab7a76d5715613d359a0e92be22716b44c1f7514bc72
0764"},"image":{"docker-manifest-digest":"sha256:14776e0fe1b95a6aab7a76d5715
613d359a0e92be22716b44c1f7514bc720764"},"type":"https://sigstore.dev/cosign/
sign/v1"},"optional":{}}}]
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$

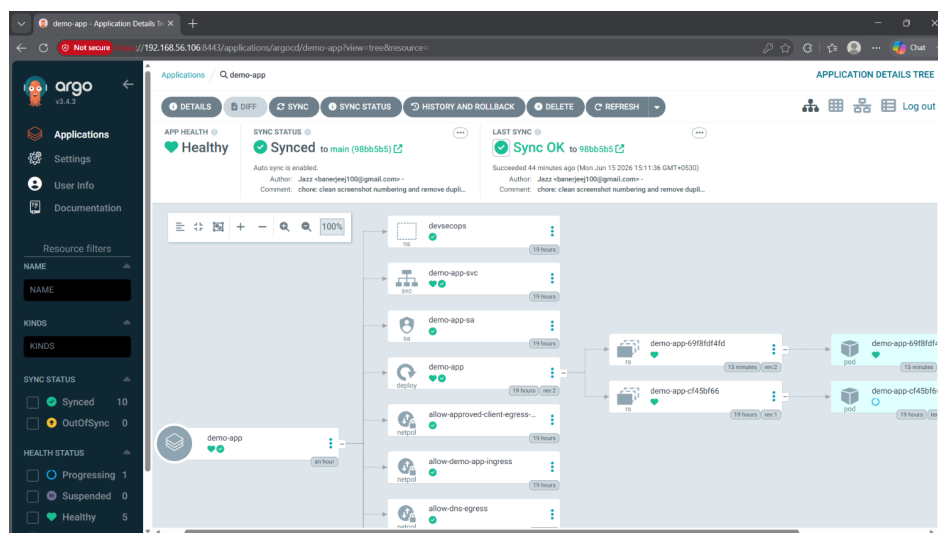
```

Evidence - Cosign verification demonstrates the image signature validation control.

11. GitOps and Deployment Threats

ArgoCD introduces a GitOps control plane where the desired Kubernetes state is pulled from Git. This reduces uncontrolled manual deployment but creates a new need to protect repository configuration, ArgoCD Application definitions, destination namespaces and sync permissions.

Threat	Scenario	Mitigation
Unauthorized manifest path	ArgoCD Application points to a folder controlled by an attacker.	Restrict allowed repos/paths and require review of Application definitions.
Deployment drift	Someone scales or modifies live workloads manually.	Enable selfHeal and prune; monitor OutOfSync state.
Overbroad ArgoCD access	User or project can deploy to any namespace or cluster.	Use ArgoCD RBAC and AppProject destination restrictions.
Unsafe manifest admitted	Manifest includes privileged pod, hostPath or broad RBAC.	Gatekeeper, Pod Security, Trivy config and RBAC review.



Evidence - ArgoCD resource tree shows GitOps-managed deployment objects.

```

jazz@ubuntu-enc x jazz@ubuntu-enc x jazz@ubuntu-enc x + - - - - -
Waiting for ArgoCD self-heal back to Git-defined replicas...
Check 1/60
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
demo-app  1/1     1            1           19h

NAME      READY   STATUS    RESTARTS   AGE
demo-app-69f8df4fd-st7p7  1/1     Running   0           32m
demo-app-c45bf66-km488    0/1     Terminating   0           19h

NAME      SYNC STATUS  HEALTH STATUS
demo-app  Synced      Healthy

Final self-healed state:
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
demo-app  1/1     1            1           19h

Git desired state:
9:  replicas: 1

ArgoCD status:
NAME      SYNC STATUS  HEALTH STATUS
demo-app  Synced      Healthy
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$

```

Evidence - ArgoCD self-healing demonstrates drift correction.

12. Kubernetes Admission and Runtime Threats

The CI/CD threat model extends into Kubernetes because a pipeline is only secure if the deployment target enforces safe workload behavior. The lab uses hardened deployment manifests, least-privilege RBAC, NetworkPolicy, Gatekeeper, Pod Security and Falco detection to reduce post-deployment risk.

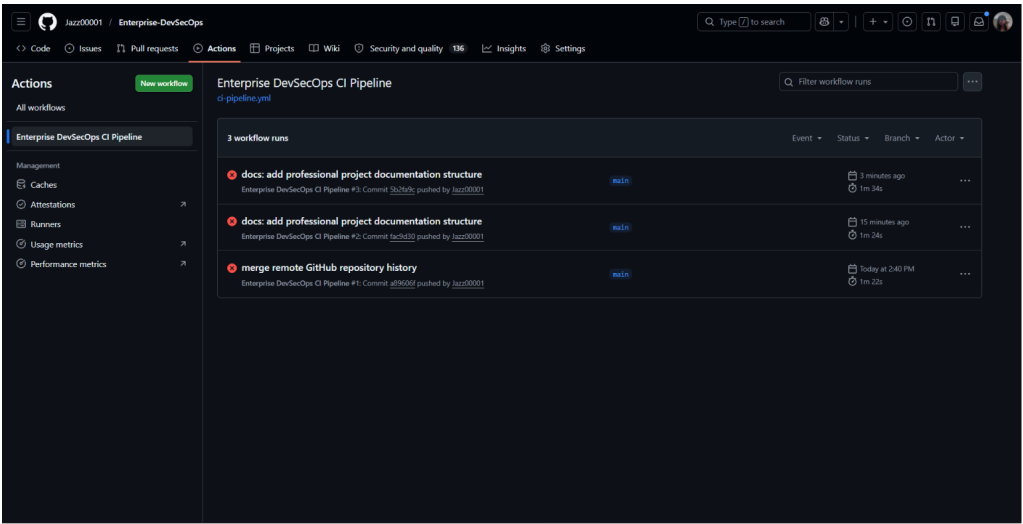
Threat	Impact	Control
Privileged pod deployed from pipeline	Potential host access and container escape pattern.	Pod Security restricted, Gatekeeper, Trivy config.
Overprivileged service account	Secret access, cluster modification or privilege escalation.	Namespace-scoped RBAC and kubectl auth can-i validation.
Open pod-to-pod networking	Lateral movement after compromise.	Default-deny NetworkPolicy and explicit allow rules.
Runtime anomaly not detected	Delayed incident response and weak evidence.	Falco, Prometheus metrics and Grafana dashboards.

Pipeline-to-runtime security principle
CI/CD controls must not stop at image build. Runtime admission and detection controls are required because a clean pipeline can still deploy dangerous Kubernetes configuration if admission policy is weak.

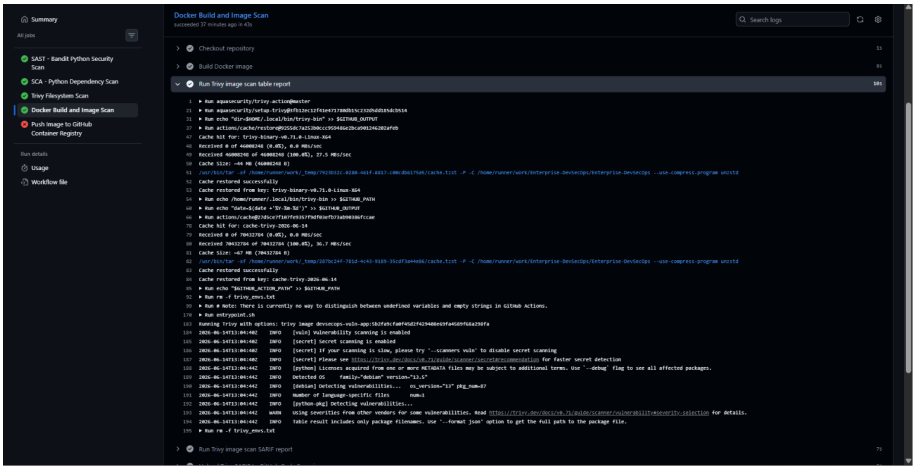
13. Evidence Mapping

The threat model is supported by lab evidence. These screenshots demonstrate that the modeled controls were implemented and observed.

Evidence	Control Demonstrated	Threat Addressed
057-github-actions-pipeline-running.png	Workflow execution and job orchestration.	Pipeline visibility and repeatable CI/CD execution.
059-bandit-job-output.png	SAST job output.	Application-level insecure coding risk.
061-pip-audit-job-output.png	Dependency scanning output.	Known vulnerable dependency risk.
064-trivy-image-scan-job.png	Image scan in CI/CD.	Container image vulnerability risk.
066-ghcr-image-pushed.png	Image published to registry.	Controlled artifact publishing.
115-cosign-signature-verified.png	Signature verification.	Artifact integrity and authenticity risk.
099-argocd-resource-tree.png	GitOps-managed resources.	Manual deployment drift and uncontrolled deployment risk.
103-argocd-self-healed-back-to-git-state.png	Drift correction.	Live-state tampering and configuration drift.



Evidence - GitHub Actions pipeline running.

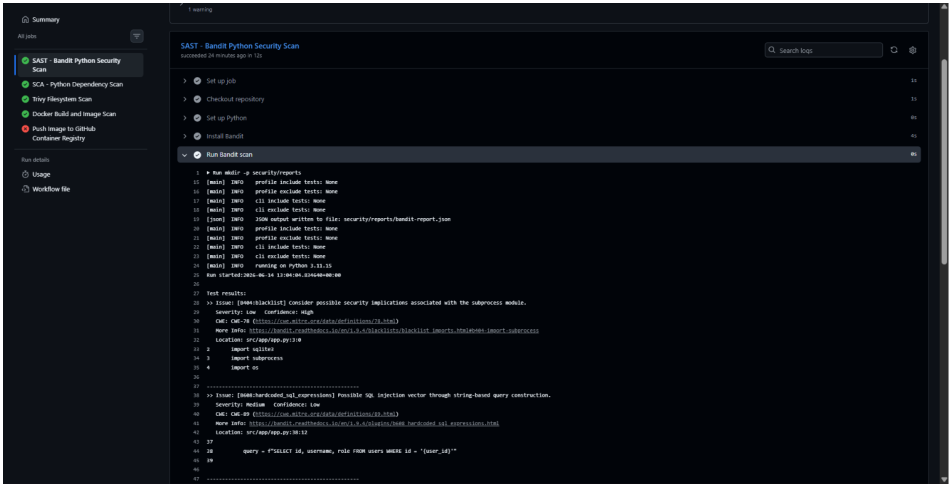


Evidence - Trivy image scan job output.

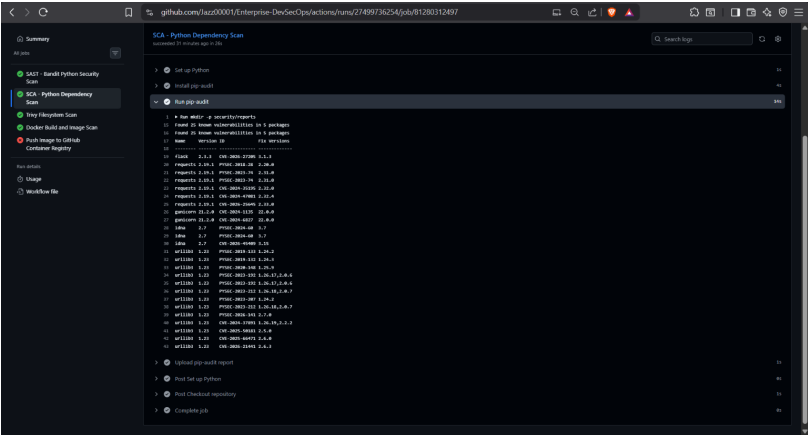
14. Security Gate Logic and Abuse Resistance

Security gates are modeled as decision points that reduce the probability of unsafe code or artifacts moving to the next pipeline stage. In the lab, several scan jobs run in observation mode with report artifacts. For production usage, selected gates should become blocking controls with documented exception handling.

Gate	What It Detects	Current Behavior	Production Recommendation
Bandit SAST	Python insecure code patterns such as shell=True and unsafe usage.	Runs and publishes report.	Fail on high-confidence high-severity findings after tuning.
pip-audit SCA	Known vulnerable Python packages.	Runs and publishes JSON report.	Fail on critical/high fixable vulnerabilities.
Trivy filesystem	Secrets, vulnerable packages and config issues.	Runs in CI/CD.	Block secrets and critical misconfigurations.
Trivy image	Container image vulnerabilities.	Runs and produces table/SARIF output.	Block critical/high fixable vulnerabilities; create exception register.
Cosign	Image signing and verification.	Implemented in supply-chain phase.	Require signature verification before deployment.
Gatekeeper	Non-compliant Kubernetes resources.	Blocks unsafe resources in cluster.	Expand policies to cover signature, digest, RBAC and securityContext requirements.



Evidence - Bandit SAST job output.



Evidence - pip-audit dependency scan output.

15. Residual Risk and Recommendations

After current lab controls, most risks are reduced but not eliminated. The remaining risks are realistic improvements that would make the lab closer to an enterprise production pipeline.

Priority	Recommendation	Reason
P1	Pin third-party GitHub Actions to commit SHA.	Reduces risk of compromised or unexpectedly changed upstream actions.
P1	Move security gates from report-only to blocking thresholds.	Prevents high-risk issues from reaching image build or deployment.
P1	Use job-level permissions instead of broad workflow-level permissions.	Limits token blast radius if a job is compromised.
P2	Use immutable image digests in Kubernetes manifests.	Prevents tag substitution and improves deployment traceability.
P2	Add signature verification admission policy.	Ensures only signed images can be deployed.
P2	Use CODEOWNERS for workflow, Dockerfile and Kubernetes manifest changes.	Forces specialist review for high-risk delivery path files.
P3	Create formal exception process for vulnerabilities.	Balances operational needs with auditability.
P3	Add dependency update automation with review.	Reduces exposure window for known vulnerable packages.

Residual Risk Statement: The current lab demonstrates strong defensive coverage. The highest residual risk is not a lack of tools; it is the possibility of gate bypass, excessive workflow permissions or unreviewed workflow changes. Production hardening should focus on enforcement, review governance and identity minimization.

16. Retest and Validation Plan

A threat model should remain testable. The following validation activities can be repeated after pipeline changes to confirm that controls still operate as intended.

Validation Test	Expected Result
Modify vulnerable code and run Bandit job.	SAST findings appear in job output and report artifact.
Add old dependency version to requirements.txt.	pip-audit flags vulnerable package and report is generated.
Build image with vulnerable base package.	Trivy image scan detects issue and produces SARIF/table output.
Attempt unsigned artifact deployment policy test.	Policy should block deployment once signature verification is enforced.
Manually scale GitOps-managed deployment.	ArgoCD detects drift and self-heals back to Git state.
Try privileged Kubernetes manifest.	Gatekeeper or Pod Security rejects it at admission.
Trigger runtime sensitive-file event.	Falco emits alert and Prometheus/Grafana shows detection metrics.

Sample retest commands

```
# Confirm workflow file permissions are explicit
grep -n "permissions:" .github/workflows/ci-pipeline.yml

# Verify local Kubernetes admission still blocks unsafe resources
kubectl apply --dry-run=server -f bad-privileged-pod.yaml

# Confirm ArgoCD restores drift
kubectl scale deployment demo-app --replicas=3 -n devsecops
kubectl get application demo-app -n argocd -w

# Confirm image signature verification evidence
cosign verify --key security/cosign/cosign.pub ghcr.io//:latest
```

17. Final Threat Model Conclusion

The Enterprise DevSecOps pipeline threat model shows how a modern application delivery workflow can be attacked and defended. It covers not only application code but also the systems that build, scan, sign, publish, deploy and monitor that code.

The lab demonstrates practical security engineering maturity through layered controls: secure code scanning, dependency analysis, image scanning, artifact signing, SBOM generation, GitOps drift correction, Kubernetes admission control and runtime detection.

From a professional security perspective, the most important lesson is that CI/CD security is an identity, integrity and governance problem. A pipeline should be treated as production infrastructure because it can create production artifacts and deploy them into runtime environments.

Final assessment
The CI/CD pipeline is suitable for portfolio demonstration and defensive security learning. For production readiness, implement strict branch protection, action pinning, blocking security gates, digest-based deployments, signature verification at admission and stronger secret governance.

Capability Demonstrated	Professional Value
Threat modeling	Shows ability to reason about attack paths, trust boundaries and risk.
DevSecOps pipeline security	Shows understanding of security gates and delivery workflows.
Supply chain security	Shows knowledge of SBOMs, signing and artifact assurance.
Kubernetes deployment protection	Shows ability to connect CI/CD security to runtime hardening.
Evidence-based reporting	Shows recruiter-ready documentation and audit-style communication.